

รายงานการวิจัยเชิงลึก: สถาปัตยกรรมและเทคโนโลยีเบื้องหลังระบบ AI Roleplay และ Character Chatbot สมัยใหม่

1. บทสรุปผู้บริหาร (Executive Summary)

อุตสาหกรรมระบบสวมบทบาทด้วยปัญญาประดิษฐ์ (AI Roleplay) และระบบตัวละครโต้ตอบ (Character Chatbot Systems) ได้วิวัฒนาการจากระบบตอบคำถามทั่วไป (Generic Q&A) ไปสู่ระบบนิเวศทางวิศวกรรมซอฟต์แวร์ที่มีความซับซ้อนสูง การเปลี่ยนผ่านนี้ต้องอาศัยการประสานเทคโนโลยีหลายมิติ ได้แก่ การบริหารจัดการหน้าต่างบริบท (Context Window Management), การดึงข้อมูลตามบริบท (Retrieval-Augmented Generation: RAG), การแคชสถานะปัญญาประดิษฐ์ (Stateful KV Caching), และการปรับแต่งโมเดลเฉพาะทาง (Domain-Specific Fine-tuning) จากการศึกษาวิเคราะห์สถาปัตยกรรมทั้งแบบเปิดเผยซอร์สโค้ด (Open-source) เช่น SillyTavern และ KoboldCpp และระบบปิด (Proprietary) เช่น Character.AI, JanitorAI และ NovelAI พบว่าหัวใจสำคัญของการสร้างประสบการณ์บทบาทสมมติที่มีความสมจริงระดับวรรณกรรม (Novel-like Roleplay) ไม่ได้ขึ้นอยู่กับขนาดของโมเดลภาษาขนาดใหญ่ (LLM) เพียงอย่างเดียว แต่ขึ้นอยู่กับ Prompt Construction Engine และ Memory Tiering Architecture ที่ทำหน้าที่แปลงโครงสร้างข้อมูลระดับสเปกตรัมของตัวละคร (Character Cards, Lorebooks, World States) ให้กลายเป็นโครงสร้างพรอมต์เชิงลำดับความสำคัญ (Prioritized Prompt Pipeline) ก่อนส่งเข้ากระบวนการประมวลผลผลลัพธ์ (Inference) รายงานฉบับนี้ทำการถอดรหัสสถาปัตยกรรมระบบอย่างละเอียด ตั้งแต่ระดับโครงสร้างไฟล์ไบนารี PNG Metadata Chunk, อัลกอริทึมการค้นหาข้อความ Aho-Corasick ใน Lorebook, การบริหารจัดการหน่วยความจำระยะยาวผ่าน Vector Database ไปจนถึงเทคนิคระดับล่างของโมเดล เช่น MixAttention, RadixAttention และการประเมินผลโมเดลด้วย RoleRM เพื่อให้วิศวกรระบบสามารถนำไปออกแบบและสร้างระบบ AI Roleplay ระดับการใช้งานจริง (Production-grade Architecture) ได้อย่างสมบูรณ์

2. การวิเคราะห์ภูมิทัศน์และสถาปัตยกรรมแพลตฟอร์ม (Platform Landscape & Architecture Comparison)

การศึกษาวិเคราะห์สถาปัตยกรรมแพลตฟอร์ม AI Roleplay และ Character Chatbot ในปัจจุบัน สามารถแบ่งออกเป็นสองตระกูลใหญ่ ได้แก่ แพลตฟอร์มฝั่งสถาปัตยกรรมเปิด/ไคลเอนต์อิสระ (Open-Architecture / Decoupled Clients) ที่แยกแยะส่วนหน้าการใช้งาน (Frontend) ออกจากส่วนประมวลผลโมเดล (Model Engine) และแพลตฟอร์มระบบปิดแบบรวมศูนย์ (Proprietary Monolithic Platforms) ในการประเมินสถาปัตยกรรมของแต่ละแพลตฟอร์ม ได้มีการจัดกลุ่มหลักฐานทางวิศวกรรมออกเป็นสามระดับ:

- **FACT:** สิ่งที่มีหลักฐานยืนยันจากซอร์สโค้ด, เอกสารทางการ หรือเอกสารงานวิจัย
- **INFERENCE:** สิ่งที่วิเคราะห์และอนุมานเชิงเทคนิคจากพฤติกรรมของ API, การตอบสนองของระบบ และการย้อนรอยวิศวกรรม (Reverse Engineering)
- **SPECULATION:** สมมติฐานเชิงสถาปัตยกรรมที่ยังไม่มีหลักฐานการทดสอบเชิงประจักษ์อย่างสมบูรณ์

มิติทางเทคนิค (Technical Dimensions)	SillyTavern [FACT]	JanitorAI [FACT/INFERENCE]	Character.AI [FACT/INFERENCE]	NovelAI [FACT]	Kindroid [INFERENCE]	KoboldCpp [FACT]
1. Architecture Style	Decoupled Client-Server Frontend	Centralized Web Application	Monolithic Cloud Distributed	Cloud-based Writing Environment	Managed SaaS Companion	Local LLM Engine & API Server
2. Frontend Tech Stack	Node.js / HTML / JS / CSS	React / Next.js Web UI	Custom Web/Mobile App	Vue.js / Custom Canvas UI	Web / React Native Mobile	C/C++ Embedded Web UI
3. Backend Engine	Node.js Express Proxy Server	Node.js Microservices	Custom C++ Serving Engine	Python / C++ Cloud Infrastructure	Cloud Orchestrator	C/C++ (ggml/llama.cpp core)
4. Model Provider Support	OpenRouter, OpenAI, Anthropic, Local	JanitorLLM, Proxy (OpenRouter/OAI)	In-house Proprietary Models	In-house Models (Kayra, Erato)	Proprietary Fine-tuned LLM	Local Models (GGUF Format)
5. Prompt Construction	Dynamic Rule-Based Pipeline	Modular Prompt Builder	System Hidden Orchestrator	Preset-based Story Context	Layered Persona Prompt	Raw Token/Template Processor
6. Memory System	Dynamic Context + Vector RAG	Context Window + Lorebook	Sliding Window + Vector RAG	Lorebook + Phrase Memory	Dynamic Vector RAG	Token Context Buffer
7. Context Management	Token-budgeted Sliding Window	Sliding Window + Custom Limit	MixAttention Sliding Window	Token-budget Insertion Engine	Sliding Window + Memory Injection	Ring Buffer KV Cache
8. Character Definition	Card V2/V3 Spec (JSON/PNG)	Form Fields / JSON Schema	Proprietary Character Definition	Prose/Attributes Definition	Structured Custom Profile	Prompt Template Definition
9. Persona System	Multi-User Profile Overlay	Dynamic User Persona Block	User Profile Setting	Author's Note / Context	User Memory Profile	Token Context Injection
10. World Information	World Info / Lorebooks	Lorebooks Integration	Hidden World Knowledge	Native Lorebook System	Contextual Lore Memory	Embedded Context Lore
11. Lorebook Engine	Aho-Corasick Keyword Trigger	Keyword Matching Engine	Vector Semantic Retrieval	Token-exact Keyword Scan	RAG Triggered Memory	Context Keyword Scanner
12. Retrieval Tech	Vector DB Extension (Chroma/Local)	Basic Vector Matching	Internal Vector Store (Vertex AI)	Deterministic Token Match	Semantic Vector Search	N/A (Direct Context Injection)
13. Conversation Storage	Local JSON / File-system	Cloud Database (PostgreSQL)	Cloud Database Store	Cloud Encrypted Storage	Cloud Database	Local SQLite / JSON

มิติทางเทคนิค (Technical Dimensions)	SillyTavern [FACT]	JanitorAI [FACT/INFERENCE] /NoSQL)	Character.AI [FACT/INFERENCE]	NovelAI [FACT]	Kindroid [INFERENCE]	KoboldCpp [FACT]
14. Long-term Memory	RAG Vector Embeddings	Context Summary / Lorebook	User Pinned Memories	Historical Context Insertion	Vectorized Memory Store	Context Sliding Window
15. Generation Settings	Full Control (Temp, Top-P, Min-P)	Moderate Control	Restricted User Controls	Extreme Control (Custom Samplers)	Limited Parameter Sliders	Full Control (Low-level Samplers)
16. Streaming Tech	Server-Sent Events (SSE)	Server-Sent Events (SSE)	WebSockets / SSE	SSE Streaming	WebSockets / SSE	HTTP SSE / Chunked Transfer
17. Token Management	Precise Tokenizer Budgeting	Automatic Token Truncation	Dynamic Server Truncation	Explicit Token Count Limits	Automatic Allocation	Precise KV Cache Allocation
18. Message Editing	In-place Tree Node Editing	In-place Message Modification	In-place Message Editing	Direct Text Buffer Editing	In-place Edit	Text Buffer Manipulation
19. Regeneration	Alternate Branch Generation	Single/Multi Regeneration	Swipable Alternates (Swipes)	In-place Re-generation	Regenerate Response	In-place Token Regeneration
20. Chat Branching	Native Tree Branching System	Linear History	Linear with Swipe Variants	Nonlinear Story Tree	Linear Conversation	State Snapshot / Save States
21. Summarization	Automated Background LLM Summarizer	Periodic History Truncation	Background Memory Synthesis	Manual/Auto Memory Summary	Dynamic Memory Summary	N/A (Requires External Client)
22. Context Compression	Token Trimming / Summarization	Context Window Truncation	MixAttention Compression	Token Budget Optimization	Semantic Compression	Context Shift / KV Cache Reuse
23. Model Switching	On-the-fly Model Swap	API Endpoint Switching	Locked System Model	Model Selection (Kayra/Erato)	Locked Proprietary Engine	Model Reload via API
24. Fine-Tuning Focus	N/A (Client-side Software)	JanitorLLM Fine-tuning	Massive Multi-character RLHF	Fine-tuned Fiction Corpus	Proprietary Persona SFT	N/A (Engine Level)
25. Safety / Moderation	Zero Filter (User Controlled)	AutoMod + AWS Rekognition	Strict Safety Layer / Classifier	Minimal / Uncensored	Moderate Safety Guardrails	Zero Filter (Engine Level)
26. Cost Optimization	Client-side Free / API Usage	Free JLLM / Bring Your Own Key	High Scale In-house Optimization	Subscription Token Tiers	Subscription Tier	Local Hardware Dependent

มิติทางเทคนิค (Technical Dimensions)	SillyTavern [FACT]	JanitorAI [FACT/INFERENCE]	Character.AI [FACT/INFERENCE]	NovelAI [FACT]	Kindroid [INFERENCE]	KoboldCpp [FACT]
			s			
27. Latency Target	Dependent on Model API	Dependent on Proxy/JLLM	Optimized High Throughput	Low-latency Inline Generation	Fast Interactive Streaming	Hardware Bound (Tokens/sec)

3. การวิเคราะห์เชิงเทคนิคของ SillyTavern (SillyTavern Deep Technical Analysis)

SillyTavern เป็นสถาปัตยกรรมส่วนหน้า (Frontend Architecture) แบบโอเพ่นซอร์สที่ได้รับการยอมรับว่าเป็นมาตรฐานอุตสาหกรรมสำหรับการสวมบทบาทด้วย AI โดยระบบทำงานบน Node.js Express server ที่ทำหน้าที่เป็น API Proxy และ Context Orchestration Engine

3.1 ข้อกำหนดและโครงสร้างข้อมูล Character Card Spec V2 และ V3 [FACT]

SillyTavern จัดเก็บข้อมูลตัวละครไว้ในรูปแบบไฟล์ภาพ PNG ที่มีการฝังข้อมูลแบบอักษรข้อความ (tEXt chunk) ตามมาตรฐาน PNG Specification โดยใช้อัลกอริทึม Base64 Encoded JSON string ในการอ่านและบันทึก

Character Card V2 (chara chunk) ใช้นามสเปซ chara ใน tEXt chunk โดยมีโครงสร้าง JSON ภายในดังนี้:

```
{
  "spec": "chara_card_v2",
  "spec_version": "2.0",
  "data": {
    "name": "Seraphina",
    "description": "An ancient, cynical mage living in a
[span_24] (start_span) [span_24] (end_span) [span_26] (start_span) [span_26] (end_span) secluded tower.",
    "personality": "Cynical, brilliant, secretly protective, soft-spoken.",
    "scenario": "The user stumbles into Seraphina's sanctuary while fleeing a storm.",
    "first_mes": "*Seraphina doesn't look up from her grimoire as the heavy oak door creaks open.* \"You're tracking mud on the runes, stranger.\"",
    "mes_example": "<START>\n{{user}}: Can you help me?\n{{char}}: *She sighs, closing the book with a heavy thud.* \"Help is a costly word. What do you offer in return?\"",
    "creator_notes": "Optimized for Claude-3.5-Sonnet and Llama-3 70B models.",
    "system_prompt": "Write {{char}}'s next reply in a detailed, descriptive novel style.",
    "post_history_instructions": "Maintain an atmosphere of dark fantasy and tension.",
```

```

    "alternate_greetings": [
      "*Rain pours outside as Seraphina brews a glowing blue potion.*
    \nState your business.\n"
    ],
    "tags": ["Fantasy", "Mage", "Tsundere"],
    "creator": "ArchMage",
    "character_version": "1.0",
    "character_book": {}
  }
}

```

Character Card V3 (ccv3 chunk) คุมมาตรฐานด้วยคีย์เวิร์ด ccv3 เพื่อรองรับโครงสร้าง Lorebook แบบ Native, สินทรัพย์หลายภาษา (Multi-language assets) และ Creator Metadata ที่มีโครงสร้างระดับสูงขึ้น โดย SillyTavern รองรับการส่งออกไฟล์ที่มีทั้ง chara และ ccv3 แบบ Dual-chunk เพื่อรักษาความสามารถในการทำงานร่วมกัน (Backward Compatibility)

3.2 อัลกอริทึมการค้นหาและการเปิดใช้งาน Lorebook (World Info Engine) [FACT]

ระบบ Lorebook ของ SillyTavern ทำหน้าที่เป็นระบบความจำแบบจำแนกเงื่อนไข (Conditional Memory) โดยใช้อัลกอริทึมการค้นหาข้อความหลายรูปแบบพร้อมกัน (Multi-pattern Matching Algorithm) เช่น Aho-Corasick Automaton ในการสแกนหาข้อความค้นหา (Keys) จากบริบทบทสนทนาล่าสุด ขั้นตอนการทำงานของ Lorebook Engine มีลำดับดังนี้:

1. **Context Scanning Depth:** ระบบสแกนข้อความย้อนหลังจากข้อความล่าสุดไปจนถึงความลึก (Depth) ที่กำหนด
2. **Key Matching Logic:**
 - Constant Match: แทรกข้อมูลเข้าสู่ Prompt โดยเสมอโดยไม่สนใจคีย์เวิร์ด
 - Selective Match (AND Logic): ทำงานเมื่อตรวจพบทั้ง Primary Key และ Secondary Key พร้อมกันในข้อความ
 - Regex Match: ใช้การสแกนด้วย Regular Expressions ตามสเปกตรัม PCRE
3. **Recursive Scanning:** เมื่อ Lorebook Entry A ถูกเปิดใช้งาน (Activated) เนื้อหาภายใน Entry A จะถูกนำไปสแกนรอบใหม่ เพื่อค้นหาคีย์เวิร์ดของ Entry B ซึ่งกระบวนการนี้สามารถวนลูปได้ตามค่า recursion_depth ที่ตั้งไว้
4. **Insertion Position & Order:** รายการที่ผ่านการคัดเลือกจะถูกนำไปจัดเรียงตามค่า Priority และแทรกตามตำแหน่งที่กำหนด เช่น @depth (แทรกย้อนหลังจากข้อความล่าสุด N บรรทัด) หรือแทรกไว้ส่วนบนสุดของ Context

3.3 ตัวอย่างการประกอบพรอมต์ (Hypothetical Prompt Construction Trace) [FACT/INFERENCE]

พิจารณาสถานการณ์ที่ User ส่งข้อความ: "I offer you this ancient crystal I found in the ruins." ลำดับการประกอบของค้ประกอบภายใน Context Window ขนาด 8,192 Tokens ของ SillyTavern จะเรียงลำดับจากบนลงล่างดังนี้:

- **Position 1 (System Prompt / Base Instruction):** Roleplay as Seraphina. Write in dark fantasy prose style. Use markdown for actions.
- **Position 2 (Character Description):** Name: Seraphina\nPersonality: Cynical, brilliant, secretly protective, soft-spoken.\nDescription: An ancient, cynical mage living in a

- secluded tower.
- **Position 3 (Scenario):** Scenario: The user stumbles into Seraphina's sanctuary while fleeing a storm.
- **Position 4 (Lorebook Injections - Constant & Triggered):** [Lorebook Entry: Ancient Crystal] (Triggered by keyword "crystal")\nThe Crystal of Aethelgard absorbs ambient mana and glows with a pale azure light. Highly valued by mages.
- **Position 5 (Example Dialogues - mes_example):** <START>\n{{user}}: Can you help me?\n{{char}}: *She sighs, closing the book with a heavy thud.* "Help is a costly word. What do you offer in return?"
- **Position 6 (Chat History - Sliding Window):** User: *I stagger into the tower, soaking wet from the storm.*\nSeraphina: *Seraphina doesn't look up from her grimoire as the heavy oak door creaks open.* "You're tracking mud on the runes, stranger."
- **Position 7 (Author's Note - @depth 2):** [Author's Note: Focus on Seraphina's intrigue towards the magical artifact.]
- **Position 8 (Recent Chat History Continued):** User: I offer you this ancient crystal I found in the ruins.
- **Position 9 (Post-History Instructions / Jailbreak):** Maintain an atmosphere of dark fantasy and tension. Always remain in character as Seraphina. Never answer as {{user}}.

4. การวิเคราะห์เชิงลึกของ JanitorAI (JanitorAI Deep Analysis)

JanitorAI เป็นแพลตฟอร์มสวมบทบาทที่เติบโตอย่างรวดเร็ว โดยออกแบบสถาปัตยกรรมในลักษณะ Decoupled API Proxy Frontend ควบคู่ไปกับการให้บริการโมเดลภายในองค์กรชื่อ JanitorLLM (JLLM) การรับส่งข้อมูลของ JanitorAI เริ่มจาก JanitorAI Frontend UI ซึ่งทำหน้าที่ส่งผ่าน API Requests ไปยังสองเส้นทางหลัก ได้แก่ Internal JanitorLLM Serving Cluster สำหรับผู้ใช้ทั่วไป หรือส่งผ่านไปยัง External Proxy API (เช่น OpenRouter หรือ OpenAI Direct Endpoints) เมื่อผู้ใช้กำหนด API Key ของตนเอง

4.1 การจัดเก็บและการจัดการบริบทสนทนา [FACT/INFERENCE]

JanitorAI ได้ทำการขยายขีดจำกัดความยาวของ Saved Custom Prompt สูงสุดถึง 500,000 ตัวอักษร (500K characters) เพื่อรองรับการใส่คำสั่งเชิงลึกและตัวละครที่มีโครงสร้างซับซ้อน ในเชิงโครงสร้าง โมเดล JanitorLLM เคยเผชิญปัญหาการเกิด Repetition Loops (การสร้างข้อความซ้ำซาก) ทีมพัฒนาได้ปรับปรุงอัลกอริทึมการสุ่มตัวอย่างและการควบคุม Penalty ทำให้สถิติการติดลูปตามการทดสอบลดลงอย่างมีนัยสำคัญ จากระดับ 6/64 เหลือเพียง 2/64 ในชุดการทดสอบมาตรฐาน โดยไม่สูญเสียคะแนนด้านตรรกะใน Benchmark ระบบความปลอดภัยของ JanitorAI ประกอบด้วยการทำงานประสานกันของ AutoMod, DynoBot, และ AWS Rekognition ในการคัดกรองเนื้อหาประเภทรูปภาพ ควบคู่ไปกับการยืนยันตัวตนด้วยบัตรประชาชน (ID Verification) สำหรับการเข้าถึงเนื้อหาจำกัดอายุ 18+

5. สถาปัตยกรรมระบบปิดและ Character.AI (Closed-Source Systems & Character.AI Architecture)

Character.AI เป็นแพลตฟอร์มระบบปิดที่รองรับปริมาณผู้ใช้งานระดับมหาศาล (Massive Concurrent Scale) การประมวลผลข้อมูลของ Character.AI ไม่ได้พึ่งพาการสแกนค่าแบบดั้งเดิม แต่ใช้เทคนิคการ

ประมวลผลการคำนวณขั้นสูงระดับ Kernel และสถาปัตยกรรมโมเดล
กระบวนการประมวลผลคำขอเริ่มจาก Incoming Chat Query เข้าสู่ Sticky Session Router ซึ่งดึงข้อมูล
แคชจาก RadixAttention Tree Cache ที่เก็บบน Host System Memory หากตรวจพบ Context Match
(ซึ่งมีอัตราความสำเร็จสูงถึง 95%) ระบบจะข้ามขั้นตอน Prefill และส่งข้อมูลไปยัง Native Int8 Tensor
Core Inference Engine ทันที

5.1 นวัตกรรมระดับล่างของ Character.AI Inference Stack [FACT]

จากเอกสารทางเทคนิคของ Character.AI การให้บริการโมเดลแก่ผู้ใช้งานหลักสิบล้านคนต่อวันภายใต้ข้อ
จำกัดด้านต้นทุน มีการใช้เทคนิคสำคัญดังนี้:

1. **MixAttention Architecture:** การปรับแต่งสถาปัตยกรรม Attention ที่ผสมผสานระหว่าง Sliding Window Attention (เก็บเฉพาะ KV Cache ของ Token ล่าสุดไว้บน GPU Memory) ร่วมกับการแชร์ KV Cache ข้าม Layer (Cross-layer Cache Sharing) ทำให้ลดขนาด KV Cache รวมได้มากถึง 20 เท่า โดยไม่สูญเสียคุณภาพของเนื้อหาอย่างมีนัยสำคัญ
2. **Stateful Caching & RadixAttention:** บทสนทนาใน Character.AI มีความยาวเฉลี่ยถึง 180 ข้อความ หากต้องทำ Prefill ใหม่ทุกรอบการตอบสนอง ค่าการคำนวณ $O(N^2)$ จะสูงมาก ระบบจึงใช้ Stateful Caching ร่วมกับโครงสร้างข้อมูลต้นไม้ (Tree-structured LRU Cache) ที่ดึงแนวคิดจาก RadixAttention ข้อมูล KV Cache จะถูกย้ายไปเก็บบน Host Memory (RAM ของเครื่อง Server) และดึงกลับมาผ่าน Rolling Hash ของ Prefix Tokens ระบบใช้ Sticky Sessions ในการส่งคำสั่งจากบทสนทนาเดียวกันไปยัง Server เครื่องเดิมเสมอ ส่งผลให้ทำ Cache Hit Rate ได้สูงถึง 95% และลดต้นทุนการให้บริการลงได้ถึง 33 เท่า เมื่อเทียบกับสถาปัตยกรรมมาตรฐาน
3. **Native Int8 Quantization-Aware Training (QAT):** Character.AI ไม่ได้ใช้ Post-Training Quantization (PTQ) แต่ทำการฝึกฝนโมเดล (Training) ด้วยความละเอียด Int8 Natively ทั้งในส่วน Weights, Activations และ KV Cache ทำให้ไม่มีปัญหา Performance Mismatch และเพิ่มความเร็วในการประมวลผลด้วย Int8 Matrix Multiplication Kernels

5.2 ข้อจำกัดด้านระบบความจำของ Character.AI [FACT/INFERENCE]

แม้ว่า Character.AI จะมีความสามารถสูงในการรองรับความยาวบทสนทนา แต่ตัวโมเดลรองรับ Context Window อยู่ที่ประมาณ 8,000–9,000 Tokens เท่านั้น ข้อมูลที่เก่ากว่านั้นจะถูกตัดออกจาก Context Window แบบ Sliding Window การจดจำความจริงระยะยาวในปัจจุบันพึ่งพาฟิเจอร์ปักหมุดข้อความ (User Pinned Memories) หรือการรวบรวมสรุปบริบทบทสนทนา (Background Memory Synthesis) เข้าสู่ Prompt Injection

6. กลไกการสร้างบทบาทสมมติเชิงวรรณกรรม (How AI Generates "Novel-Like" Roleplay)

ความแตกต่างระหว่างโมเดลการตอบคำถามเชิงช่วยเหลือ (Instructional Assistant) กับโมเดลสวมบทบาทเชิงวรรณกรรม (Novel-Like Roleplay Engine) เกิดขึ้นจากการปรับแต่งโครงสร้างทางภาษาและกลไกพฤติกรรมในระดับโมเดลและพรอมต์
กระบวนการเปลี่ยนคำตอบสั้นให้เป็นบทบรรยายเริ่มจาก Direct User Input Prompt ซึ่งถูกห่อหุ้มด้วย Dynamic Prompt Context Envelope ประกอบด้วย Third-Person POV Anchor, Dialogue vs Action Filter, และ Inner Monologue Orchestrator ก่อนส่งเข้าประมวลผลด้วย High-Prose LLM Inference เพื่อสร้างผลลัพธ์สไตลีนิยาย

6.1 ปัจจัยขับเคลื่อนพฤติกรรมการเล่นระดับวรรณกรรม

การแปลงคำตอบที่แห้งแล้งให้เป็นบรรยายเชิงวรรณกรรมที่สมบูรณ์ ขึ้นอยู่กับ 8 ปัจจัยหลัก:

1. **Base Model Capabilities:** โมเดลที่ได้รับการฝึกฝนล่วงหน้า (Pre-trained) ด้วยคลังข้อมูลวรรณกรรมและนิยายคุณภาพสูง (เช่น LLaMA-3, Mistral, หรือโมเดล Kayra/Erato ของ NovelAI) จะมีความเข้าใจโครงสร้างจังหวะของเนื้อเรื่อง (Pacing) และคลังคำศัพท์ที่หลากหลายกว่าโมเดลสายรหัสหรือข้อมูลทั่วไป
2. **Fine-Tuning (SFT / Preference Optimization):** การปรับแต่งด้วยข้อมูลบทสนทนาวรรณกรรมแบบสวมบทบาทที่สอนโมเดลให้หลีกเลี่ยงรูปแบบ Assistant Artifacts (เช่น คำลงท้ายว่า "Is there anything else I can help you with?") และสร้างโครงสร้างแบบ Action-Dialogue Hybrid Format
3. **Prompt Envelope & Style Anchoring:** คำสั่งบังคับรูปแบบ เช่น การกำหนดมุมมองการเล่าเรื่อง (POV: Third-person limited), เวลา (Tense: Past/Present tense), และการสลับกฎการใช้สัญลักษณ์ Markdown (เช่น **Actions in italics** และ "Dialogue in quotes")
4. **Few-Shot / Narrative Examples:** การใส่ mes_example ในสเปก V2/V3 เพื่อสลับโครงสร้างจังหวะภาษา (Rhythm), โทนเสียง (Tone), และสไตล์การตอบสนองของตัวละครลงในบริบท
5. **Inner Monologue Extraction:** เทคนิคการสั่งให้โมเดลประมวลผล "ความคิดภายใน" (Internal Monologue) ก่อนจะแสดงออกเป็นคำพูดหรือการกระทำ เพื่อเพิ่มมิติความลึกทางอารมณ์ของตัวละคร
6. **Environmental Description Anchoring:** คำสั่งที่บังคับให้โมเดลอธิบายสภาพแวดล้อม เสียง กลิ่น และบรรยากาศรอบข้างเพื่อสร้างอารมณ์ร่วม (Atmosphere)
7. **Pacing Controls:** การปรับแต่งเพื่อชะลอจังหวะการดำเนินเรื่อง (Prevent fast forwarding) เพื่อให้บทสนทนาดำเนินไปอย่างสมจริงและมีปฏิสัมพันธ์ต่อคำตอบของผู้ใช้ในทุกรายละเอียด
8. **Sampling Parameter Tuning:** การกำหนดค่า Min-P, Temperature และ Repetition Penalty เพื่อส่งเสริมการเลือกใช้คำศัพท์ที่คาดเดาได้ยากขึ้น แต่ยังคงความสมเหตุสมผลทางไวยากรณ์

7. วิศวกรรมพรอมต์สำหรับบทบาทสมมติ (Prompt Architecture & Engineering for Roleplay)

การจัดลำดับชั้นขององค์ประกอบในพรอมต์มีผลต่อการกระจายน้ำหนัก Attention (Attention Weights) ของโมเดลภาษา โดยส่วนที่อยู่ใกล้ด้านล่างสุดของบริบท (Bottom of Context / Recency Position) มักจะมีผลกระทบต่อผลลัพธ์ทันทีมากที่สุด ในขณะที่ส่วนบนสุด (Top of Context / System Position) จะทำหน้าที่เป็นกรอบคำสั่งหลัก

7.1 พีระมิดลำดับความสำคัญขององค์ประกอบในพรอมต์ (Prompt Component Priority Pyramid)

ลำดับการจัดวางองค์ประกอบใน Context Window จากล่างขึ้นบนตามลำดับความสำคัญในการส่งผลต่อ Output:

1. **[BOTTOM] Post-History Instructions / Jailbreak:** มีความสำคัญสูงสุดและส่งผลต่อพฤติกรรมทันที (Immediate Effect)
2. **[NEAR-BOTTOM] Author's Note (@depth 2-4):** กำหนดทิศทางและอารมณ์ชั่วคราว (Dynamic Bias)
3. **[MIDDLE-LOWER] Recent Active Chat History:** บริบทบทสนทนาปัจจุบัน (Working Context)
4. **[MIDDLE] Retrieved Memories / Lorebook Injections:** ความรู้และข้อมูลเฉพาะบริบท (Contextual Facts)
5. **[MIDDLE-TOP] Few-Shot Dialogue Examples:** รูปแบบภาษาและสไตล์การพูด (Style Anchor)

6. [TOP-LOWER] World Description / Scenario: จากและสถานการณ์ของเรื่อง (Environment)
7. [TOP-MIDDLE] Character Persona / Description: อัตลักษณ์และบุคลิกตัวละครหลัก (Identity Core)
8. [TOP] System Instruction / Base Rules: กฎเกณฑ์และข้อบังคับพื้นฐานของโมเดล (System Core)

7.2 การคำนวณและจัดสรรงบประมาณ Token (Token Budgeting Algorithm)

ขนาดงบประมาณ Token สำหรับการสร้างข้อความ

$(T_{\text{span}_{163}}(\text{start_span})[\text{span}_{163}](\text{end_span})\text{max}))$ คำนวณได้จาก:

$T_{\text{max}} = T_{\text{window}} - T_{\text{generation_limit}}$

ในการประกอบพร้อม ระบบจะทำการคำนวณและตัดทอนข้อมูลตามลำดับความสำคัญ:

$T_{\text{available}} = T_{\text{span}_{165}}(\text{start_span})[\text{span}_{165}](\text{end_span})\text{max}) - (T_{\text{system}} + T_{\text{char_core}} + T_{\text{post_history}} + T_{\text{user_input}})$

1. **Fixed Overhead Reserve:** กัน Token สำหรับ System Prompt, Character Core, Post-History, และ User Input ล่วงสุด
2. **Lorebook Injection Allocation:** เต็ม Lorebook Entries ที่ผ่านการ Trigger ตามลำดับค่า Priority โดยจำกัดวงเงินไม่เกิน 20%-30% ของ T_{max}
3. **Memory/RAG Allocation:** แทรกเนื้อหาความจำที่ดึงมาจาก Vector Database
4. **Chat History Buffer:** ใช้ Token ที่เหลือทั้งหมดในการบรรจุประวัติการสนทนาย้อนหลังผ่านกลไก Sliding Window

8. สถาปัตยกรรมระบบความจำของตัวละคร (Character Memory Architecture)

ความท้าทายที่สำคัญที่สุดของระบบ AI Roleplay คือการบริหารจัดการความจำให้ก้าวข้ามขีดจำกัดของหน้าต่างบริบท (Context Window Limiting) โดยไม่สร้างความสับสนหรือข้อมูลเท็จ (Hallucinated Memories)

วงจรความจำเริ่มจาก New Chat Message เข้าสู่ Working Memory Buffer ใน RAM จากนั้นระบบจะแยกการทำงานเป็นสองสายขนานกัน ได้แก่ สายเขียนที่ใช้ Async Reflection Engine สกัดความทรงจำเข้าสู่ Episodic Memory และ Semantic Profile Store และสายอ่านที่ใช้ Memory Read Controller ดึงความจำผ่าน Vector Search Engine เข้าสู่ Context Injection Gate

8.1 ลำดับขั้นสถาปัตยกรรมความจำ 9 ระดับ (9-Tier Memory Architecture)

1. **Working Memory:** ข้อความในรอบปัจจุบันที่กำลังประมวลผลใน RAM
2. **Conversation Context:** ประวัติการสนทนาย้อนหลัง N ข้อความที่ถูกส่งเข้าไปใน Context Window
3. **Short-Term Memory:** ข้อสรุปการสนทนาช่วงสั้น (10-20 ข้อความล่าสุด) ที่ถูกสรุปด้วย LLM ขนาดเล็ก
4. **Episodic Memory:** บันทึกเหตุการณ์สำคัญที่เกิดขึ้นในอดีต (Events, Plot Progression) จัดเก็บในรูป Vector Embeddings ร่วมกับ Metadata ด้านเวลา
5. **Semantic Memory:** ข้อเท็จจริงถาวรที่ไม่ขึ้นกับเวลา เช่น ความสัมพันธ์ระหว่างตัวละคร, กฎเกณฑ์ของโลก, ความชอบ

6. **Long-Term Memory:** คลังข้อมูลดีดทั้งหมดของระบบที่ผ่านการบีบอัดและทำ Indexing
7. **Character Memory:** ความทรงจำและประสบการณ์เฉพาะตัวของฝั่ง {{char}}
8. **World Memory:** ข้อมูลประวัติศาสตร์ สภาพภูมิศาสตร์ และสถานะของโลกบทบาทสมมติ
9. **User Memory:** ข้อเท็จจริงเกี่ยวกับตัวผู้ใช้ ({{user}}) เช่น ชื่อ อาชีพ ประวัติความเป็นมา

8.2 กระบวนการดึง แก้ไข และจัดการความขัดแย้งของความจำ (Memory Reconciliation Engine)

```
{
  "memory_id": "mem_8921a",
  "subject": "User",
  "predicate": "possesses",
  "object": "Ancient Blue Crystal",
  "confidence_score": 0.95,
  "timestamp": 1718900123,
  "recency_decay_factor": 0.98,
  "status": "ACTIVE",
  "supersedes": null
}
```

เมื่อผู้ใช้ส่งข้อมูลใหม่ที่ขัดแย้งกับความทรงจำเดิม (เช่น เดิมเคยระบุว่า "เกลียดเวทมนตร์" แต่ต่อมาบอกว่า "กำลังศึกษาวิชาเวทมนตร์") ระบบ Conflict Resolution Pipeline จะทำงานดังนี้:

1. **Fact Extraction:** โมเดลสกัดค่ากล่าวอ้างใหม่ในรูปโครงสร้าง Triplet (Subject, Predicate, Object)
2. **Similarity Lookup:** ค้นหาความทรงจำที่มี Subject และ Predicate ใกล้เคียงกันใน Vector DB
3. **Contradiction Detection:** ใช้ LLM Evaluator ตรวจสอบว่าข้อความใหม่ขัดแย้งกับข้อความเดิมหรือไม่
4. **Supersede Update:** หากขัดแย้ง ข้อความเดิมจะถูกเปลี่ยนสถานะเป็น SUPERSEDED และเขียนคีย์อ้างอิงไปยัง memory_id ใหม่ เพื่อให้ระบบเก็บ History ไว้โดยไม่นำข้อความเก่ามารวมประมวลผลใน Prompt

9. ระบบ Vector Database และ RAG สำหรับ Roleplay (Vector Database & RAG in Roleplay Systems)

การนำ Retrieval-Augmented Generation (RAG) มาใช้ในระบบสวมบทบาทที่มีความแตกต่างจากระบบ Enterprise Search อย่างสิ้นเชิง เนื่องจาก RAG สำหรับ Roleplay ต้องให้น้ำหนักกับความสดใหม่ของข้อมูล (Recency), ระดับความสัมพันธ์ทางอารมณ์ (Emotional Valence) และความสอดคล้องของเส้นเรื่อง (Narrative Flow) มากกว่าความถูกต้องทางเอกสารเพียงอย่างเดียว

9.1 การคำนวณคะแนนการดึงข้อมูลความจำ (Composite Retrieval Scoring Formula)

คะแนนสุดท้าย ($S_{\text{retrieval}}$) ของความทรงจำแต่ละชิ้น คำนวณจากสมการถ่วงน้ำหนักหลายมิติ:
$$S_{\text{retrieval}} = w_1 \cdot S_{\text{cosine}} + w_2 \cdot S_{\text{recency}} + w_3 \cdot S_{\text{importance}}$$

โดยที่:

- S_{cosine} คือ ค่าความคล้ายคลึงเชิงสเปกตรัมของ Vector Embedding ระหว่าง Query กับ Memory Chunk
- $S_{\text{recency}} = e^{-\lambda \cdot (T_{\text{current}} - T_{\text{memory}})}$ คือ ค่าเสื่อมถอยตามเวลา (Exponential Time Decay)
- $S_{\text{importance}}$ คือ คะแนนความสำคัญของเหตุการณ์ (1-10) ที่ประเมินโดย LLM ขณะบันทึกความจำ

9.2 เปรียบเทียบกลยุทธ์การจัดการบริบทความจำ

กลยุทธ์ (Strategy)	จุดแข็ง (Strengths)	จุดอ่อน (Weaknesses)	ค่าใช้จ่าย Token (Token Cost)	เหมาะกับสถานการณ์ (Best Use Case)
Pure Context Window	ความสอดคล้องสูงมาก โมเดลเห็นข้อความจริง	จำกัดตามขนาด Window เกิดปัญหา forgetting	สูงมาก ($O(N)$ เติบโตตามความยาว)	การสนทนารอบสั้น ความยาวต่ำกว่า 20 Messages
Summarization Only	ประหยัด Token รักษาภาพเส้นเรื่องรวม	เสียรายละเอียดเชิงลึก อารมณ์ความรู้สึกถูกตัดทอน	ต่ำถึงปานกลาง	การสรุปภาพรวมของ Session ที่ยาวนาน
Vector RAG Only	ดึงข้อมูลเก่าในอดีตได้อย่างแม่นยำ	บริบทขาดความต่อเนื่อง (Fragmented Context)	ปานกลาง	การอ้างอิงข้อเท็จจริงในอดีตไกลๆ
Hybrid Memory Architecture	มีทั้งความต่อเนื่อง ข้อเท็จจริงแม่นยำ และประหยัด Token	สถาปัตยกรรมซับซ้อน พึ่งพา Multi-LLM Calls	สมดุลที่สุด (Optimized Allocation)	ระบบ Production Roleplay ระดับสูง

10. การจัดการและจัดเก็บประวัติการสนทนา (Conversation Storage & Schema)

การออกแบบระบบจัดเก็บข้อมูลสำหรับระบบ AI Roleplay ต้องรองรับโครงสร้างการแตกกิ่งก้านของบทสนทนา (Conversation Tree Branching), การแก้ไขข้อความย้อนหลัง (Message Editing), การย้อนกลับ (Rollback) และการปรับเปลี่ยนตั้งค่าพารามิเตอร์แบบรายข้อความ โครงสร้างกิ่งก้านสนทนาเริ่มจาก Root Node (ข้อความแรก) ส่งต่อไปยัง User Choice Node ซึ่งสามารถแตกกิ่งออกเป็น AI Response Alt A และ AI Response Alt B ขนานกันไป และพัฒนาเป็น User Branch A และ User Branch B ต่อเนื่องไปเรื่อยๆ

10.1 โครงสร้างสคีมาฐานข้อมูลเชิงสัมพันธ์และเอกสาร (Relational & Document Hybrid Schema)

ในการใช้งานจริง ข้อมูลสถานะและโครงสร้างกิ่งก้านสนทนาจะถูกจัดเก็บด้วย Relational Database (เช่น PostgreSQL) หรือ Document Store (เช่น MongoDB) ร่วมกับ Vector Database:

```
-- Table: Conversations
CREATE TABLE conversations (
  conversation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  character_id UUID NOT NULL,
  current_node_id UUID,
  world_state JSONB DEFAULT '{} '::jsonb,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Table: Conversation_Nodes (Tree Structure)
CREATE TABLE conversation_nodes (
    node_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID REFERENCES conversations(conversation_id) ON
DELETE CASCADE,
    parent_node_id UUID REFERENCES conversation_nodes(node_id),
    sender_type VARCHAR(20) NOT NULL, -- 'user', 'character', 'system'
    content TEXT NOT NULL,
    generation_settings JSONB DEFAULT '{} '::jsonb, -- Temp, Top_P,
Model used
    metadata JSONB DEFAULT '{} '::jsonb, -- Token count, Latency,
Evaluation score
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Index for fast tree traversal
CREATE INDEX idx_node_parent ON conversation_nodes(parent_node_id);
CREATE INDEX idx_conv_user ON conversations(user_id);
```

11. การบริหารจัดการหน้าต่างบริบท (Context Window Management)

เมื่อบทสนทนามีขนาดใหญ่เกินกว่าความจุของหน้าต่างบริบท (เช่น บทสนทนาสะสม 100,000 Tokens บนโมเดลที่รองรับได้เพียง 32,000 Tokens) ระบบจำเป็นต้องใช้อัลกอริทึม Context Allocation Budgeting ในการบีบอัดและเลือกสรรข้อมูล

ตัวอย่างการจัดสรรงบประมาณใน Context Window ขนาด 32,000 Tokens:

- System & Character Core (Fixed): 2,000 Tokens (6.25%)
- Active RAG / Lorebook Memory: 4,000 Tokens (12.50%)
- Hierarchical Summaries (Recursive): 4,000 Tokens (12.50%)
- Recent Sliding Window Chat History: 20,000 Tokens (62.50%)
- Generation Output Buffer Reserve: 2,000 Tokens (6.25%)

11.1 อัลกอริทึมการย่อสรุปแบบลำดับชั้น (Hierarchical Recursive Summarization)

กระบวนการย่อสรุปเริ่มจากประวัติการสนทนาส่วนแรก (Messages 1..50) ถูกส่งเข้า LLM Summarizer เพื่อสร้างเป็น Summary Chunk A (500 tokens) และประวัติส่วนถัดมา (Messages 51..100) ถูกสร้างเป็น Summary Chunk B (500 tokens) จากนั้นทั้งสอง Chunk จะถูกนำมารวบรวมและย่อสรุปเข้าเป็น Global Narrative Summary (1,000 tokens) เพื่อใช้เป็นบริบทภาพรวม

1. **Chunk Generation:** เมื่อ Chat History บานออกเกิน N ข้อความ ข้อความชุดเก่าที่สุดจะถูกส่งเข้าโมเดลขนาดเล็กเพื่อสร้าง Summary Chunk
2. **Recursive Merging:** เมื่อมี Summary Chunks หลายชิ้น ระบบจะนำ Summary Chunks เหล่านั้นมารวมและย่อสรุปเข้าเป็น Global Narrative Summary

3. **Sliding Context Window Shift:** ข้อความดั้งเดิมที่ถูกย่อสรุปแล้วจะถูกปลดออกจาก Active Context Window บน GPU แต่จะยังคงถูกสลับไว้ใน Database และ Vector Store สำหรับ RAG Retrieval

12. การจัดการสถานะของโลกและตัวละคร (World & Character State Management)

ระบบ AI Roleplay และ RPG ขั้นสูงต้องการกลไกการติดตามสถานะที่แน่นอน (Deterministic State Tracking) เพื่อป้องกันปัญหาโมเดลลืมข้อมูลสำคัญ เช่น สถานะพลังชีวิต (HP), สิ่งของในกระเป๋า (Inventory), หรือความสัมพันธ์

12.1 โครงสร้างข้อมูลและการสกัดสถานะแบบอัตโนมัติ (Automated JSON State Extraction)

ระบบจะกำหนดสคีมา JSON สำหรับสถานะของโลกและตัวละครไว้ล่วงหน้า:

```
{
  "character_state": {
    "name": "Seraphina",
    "emotional_state": "Intrigued but cautious",
    "affinity_score": 45,
    "current_location": "Mage Tower - Library",
    "inventory": ["Grimoire of Aethelgard", "Azure Mana Potion"]
  },
  "world_state": {
    "time_of_day": "Night",
    "weather": "Heavy Rainstorm",
    "active_quest": "Identifying the Ancient Crystal"
  }
}
```

ขั้นตอนวงจรการอัปเดตสถานะ (State Update Lifecycle):

1. ผู้ใช้ส่งข้อความดำเนินการ
2. โมเดลภาษาหลักสร้างข้อความตอบสนองสมทบ
3. Background State Extractor Model (หรือ Function Calling Module) อ่านข้อความสมทบบทล่าสุด และเปรียบเทียบกับ JSON State ดั้งเดิม
4. Extractor ส่งคำสั่ง JSON Patch เพื่ออัปเดตค่าใน Database แบบ Asynchronous ก่อนเริ่มรอบการสนทนาถัดไป

13. การรักษาความสอดคล้องของตัวละครและโลก (Character & World Consistency Systems)

ปัญหาการหลุดบทบาท (Character Drift) หรือการที่ตัวละครสูญเสียอัตลักษณ์เมื่อการสนทนาวานานขึ้น เกิดขึ้นเนื่องจากน้ำหนักการประมวลผลของโมเดลถูกเบี่ยงเบนไปยังประวัติการสนทนาล่าสุด (Recency Bias) มากกว่าคำสั่งใน System Prompt

13.1 กลไกป้องกันการหลุดบทบาท (Anti-Drift Techniques)

1. **Role-Aware Reasoning (RAR):** จากงานวิจัยล่าสุด โมเดลได้รับการฝึกให้ประมวลผลกระบวนการคิดระดับลึกที่สอดคล้องกับบุคลิกตัวละครก่อนตอบสนอง เช่น การวิเคราะห์ว่า "ด้วยบุคลิกที่เป็นคนหวาดระแวง {{char}} ควรจะแสดงปฏิกิริยาอย่างไรต่อข้อเสนอของ {{user}}?" ก่อนที่จะสร้างคำพูดจริง
2. **Dynamic Identity Re-anchoring:** การลงแท็กสลับอัตลักษณ์ไว้ที่ตำแหน่งความสำคัญสูงสุดย้อนหลังจากข้อความล่าสุด ผ่าน Author's Note หรือ Post-History Instructions
3. **Negative Constraint Enforcement:** การตั้งค่าสั่งห้ามที่ชัดเจนในระดับ System Prompt เช่น "Never speak in a polite assistant tone. Never apologize to {{user}}."

13.2 การรักษาความสอดคล้องของโลก (Canon Enforcement)

กระบวนการตรวจสอบความสอดคล้องเริ่มจาก Raw Generated Text Candidate ถูกส่งเข้าสแกนผ่าน World Info / Lorebook Verification หากผ่านการตรวจสอบว่าสอดคล้องกับเนื้อเรื่องหลัก (Canon) จะส่งต่อไปยัง Post-Processing แต่หากพบจุดขัดแย้ง ระบบจะทำการส่งข้อความกลับไปสร้างใหม่ (Re-prompt) ทันที

1. **Deterministic Lorebook Priority:** กำหนดให้ข้อมูลใน Lorebook มีลำดับชั้นการตัดสินใจสูงสุดเหนือข้อมูลที่โมเดลจินตนาการขึ้นมาเอง
2. **Causal Timeline Tracking:** จัดเก็บลำดับเหตุการณ์สำคัญ (Timeline Events) ในฐานข้อมูล และส่งเฉพาะไทม์ไลน์ที่เกี่ยวข้องกับฉากปัจจุบันเข้าไปในบริบท
3. **Rule Verifier Verification:** ในระบบ RPG ขั้นสูง โมเดลตรวจเงื่อนไขจะทำการตรวจสอบว่าการกระทำของตัวละครละเมิดกฎของโลกหรือไม่ (เช่น การใช้เวทมนตร์ที่ต้องใช้ Mana ในขณะที่ Mana หมด) หากละเมิด ระบบจะสั่ง Regeneration พร้อมเพิ่มคำสั่งแก้ไขข้อผิดพลาดทันที

14. การปรับแต่งโมเดลสำหรับ AI Roleplay (Fine-Tuning Strategies & Training Data Research)

การปรับแต่งโมเดลภาษาเพื่อให้มีความสามารถในการสวมบทบาทขั้นสูง อาศัยกระบวนการเรียนรู้หลังการฝึกฝน (Post-training Pipeline) เริ่มจาก Raw Pre-trained LLM ผ่านการทำ Supervised Fine-Tuning (SFT) และนำไปปรับแต่งความพึงพอใจด้วย Direct Preference Optimization (DPO) จนได้เป็น Roleplay Model ที่สมบูรณ์

14.1 โครงสร้างข้อมูลสำหรับการปรับแต่งโมเดล (SFT Data Format)

ชุดข้อมูล SFT คุณภาพสูงต้องมีการจัดรูปแบบในลักษณะ Multi-turn Conversation พร้อมสลับ System Persona และ Character Card ไว้ชัดเจน:

```
{
  "messages": [
    {
      "role": "system",
      "content": "Yo[span_58] (start_span) [span_58] (end_span)u are roleplaying as Seraphina. Description: Cynical mage. Style: Novelistic prose."
    },
    {
      "role": "user",
```

```
"c[span_212] (start_span) [span_212] (end_span) [span_214] (start_span) [span_214] (end_span) ont [span_59] (start_span) [span_59] (end_span) ent": "Who are you and what is this place?"
},
{
  "role": "assistant",
  "content": "*Seraphina narrows her eyes, setting down her quill with deliberate slowness.* \"I should be asking you that question. This is my sanctuary, and you are currently trespassing.\""
```

14.2 การสังเคราะห์และงานวิจัยชุดข้อมูล (Dataset Research)

คุณภาพของโมเดล AI Roleplay ขึ้นอยู่กับความหลากหลายและระดับความประณีตของชุดข้อมูลที่ใช้ฝึกฝน จากการศึกษางานวิจัยล่าสุด เช่น CoSER (ICML 2025) และ Crab (ACL 2025) พบว่ากระบวนการสังเคราะห์ข้อมูล (Data Synthesis) มีขั้นตอนสำคัญดังนี้:

- 1. **CoSER Dataset:** รวบรวมตัวละคร 17,966 ตัว จากวรรณกรรมชื่อดัง 771 เล่ม นำมาสกัดประวัติเหตุการณ์สำคัญในชีวิต และความคิดภายใน เพื่อนำมาสร้างบทสนทนาการสวมบทบาทตามบริบทเดิม (Given-Circumstance Acting: GCA)
- 2. **Crab Dataset:** ชุดข้อมูลสวมบทบาทที่ใหญ่ที่สุดซึ่งมีการแท็กข้อมูลระดับสูง เช่น สภาวะอารมณ์ (Emotional Nuances), หัวข้อบทสนทนา (Tagged Topics), และบริบทสภาพแวดล้อม
- 3. **Quality Filtering Rules:** จัดข้อความซ้ำซ้อนผ่าน Deduplication Algorithms และกรองข้อความที่มีคำตอบรับแบบ Assistant ออกอย่างเด็ดขาด

15. การคัดเลือกโมเดลและพารามิเตอร์การสุ่มตัวอย่าง (Model Selection & Sampling Parameters)

การเลือกโมเดลและการตั้งค่าพารามิเตอร์ส่งผลโดยตรงต่อคุณภาพภาษา ความสมจริง และความเร็วในการประมวลผล

15.1 ตารางเปรียบเทียบประเภทโมเดลสำหรับ AI Roleplay

ประเภทโมเดล (Model Family)	ตัวอย่างโมเดล	ความสามารถเชิงวรรณกรรม	การปฏิบัติตามคำสั่ง	ความเร็ว / Latency	ต้นทุน (Cost)
Frontier Closed LLMs	Claude-3.5-Sonnet, GPT-4o	สูงมาก (รายละเอียดดีเยี่ยม)	สูงมาก	ปานกลางถึงเร็ว	สูงมาก (\$10.00–\$30.00 / 1M)
Cost-Optimized APIs	DeepSeek-V3, Gemini Flash	สูงมาก (ภาษาสั้นไหล)	สูงมาก	เร็วมาก	ต่ำมาก (\$0.14–\$0.28 / 1M)
Specialized Fiction LLMs	NovelAI Kayra / Erato	สูงมากที่สุด (สไตล์นิยายแท้)	ปานกลาง (เน้นเนื้อเรื่อง)	เร็วมาก	โมเดลเฉพาะแพลตฟอร์ม

ประเภทโมเดล (Model Family)	ตัวอย่างโมเดล	ความสามารถเชิง วรรณกรรม	การปฏิบัติตาม คำสั่ง	ความเร็ว / Latency	ต้นทุน (Cost)
Open-Source Fine-Tuned	Llama-3-8B-RP , Unsloth Fine-tunes	ปานกลางถึงสูง	ปานกลาง	ขึ้นกับ Hardware	ฟรี (Self-hosted)

15.2 พารามิเตอร์การสุ่มตัวอย่าง (Sampling Parameters)

กระบวนการเลือก Token เริ่มจาก Raw Unnormalized Logits ถูกปรับสเกลด้วย Temperature Scaling จากนั้นกรองคำขยะด้วย Min-P Filter นำไปลงโทษซ้ำด้วย Repetition Penalty และแปลงเป็นความน่าจะเป็นผ่าน Softmax สำหรับการสุ่มเลือก Token

- **Temperature (0.8 - 1.2):** ควบคุมความสุ่มของการเลือก Token
- **Min-P (0.05 - 0.10):** กรอง Token ที่มีความน่าจะเป็นน้อยกว่า $P_{\text{max}} \times \text{Min-P}$ ออก ช่วยตัดคำขยะได้อย่างแม่นยำขณะเปิดค่า Temperature สูง
- **Repetition Penalty (1.05 - 1.15):** ลงโทษ Token ที่เคยปรากฏขึ้นมาแล้ว เพื่อป้องกันปัญหาติดลูบการพูดซ้ำประโยคเดิม

16. ไพพ์ไลน์การสร้างข้อความ การประมวลผล และสถาปัตยกรรมหลายโมเดล (Inference Pipeline, Post-Processing & Multi-Model Agents)

ลำดับการทำงานตั้งแต่วินาทีที่ผู้ใช้ส่งข้อความ ไปจนถึงการแสดงผลลัพท์บนหน้าจอประกอบด้วย 7 ขั้นตอนหลัก:

1. **Intent & State Analysis:** สกัดเป้าหมายและอารมณ์ของผู้ใช้
2. **Lorebook Scanning:** Aho-Corasick Keyword Scan จากข้อความล่าสุด
3. **RAG Retrieval:** Cosine Search ใน Vector DB ดึง Episodic Memory
4. **Context Assembly:** จัดสรร Token Budget และประกอบพร้อมต์ตามลำดับ Priority
5. **Model Inference:** ส่ง Prompt เข้า LLM Engine ผ่าน Streaming API (SSE)
6. **Output Parsing:** สกัด JSON State และตรวจสอบกฎความปลอดภัย
7. **Memory Update:** เขียนความทรงจำใหม่ลง Vector DB และ Database

16.1 สถาปัตยกรรมหลายโมเดล (Multi-Model Agent Layout)

การใช้ LLM เพียงตัวเดียวในการทำทุกหน้าที่ก่อให้เกิดปัญหาความล่าช้า (High Latency) สถาปัตยกรรมระดับ Production จึงแบ่งหน้าที่ด้วยโมเดลหลายขนาด:

- **Model A (Main Roleplay Engine):** ใช้โมเดลขนาดใหญ่ (เช่น Claude-3.5-Sonnet หรือ DeepSeek-V3) สำหรับการสร้างบทสนทนาหลัก
- **Model B (Memory Extractor):** ใช้โมเดลขนาดเล็ก (เช่น Llama-3-8B) สรุปความทรงจำย่อยๆ แบบ Asynchronous
- **Model C (State Extractor):** ใช้โมเดลที่เชี่ยวชาญด้าน Structured Output (JSON Mode) อัปเดตสถานะของโลก
- **Model D (Moderation Guardrail):** ใช้โมเดลการจัดประเภทขนาดเล็ก ตรวจสอบเนื้อหาอันตรายก่อนแสดงผล

17. สถาปัตยกรรม AI Agent / NPC และการเล่าเรื่องราว

(Autonomous AI Agent/NPC & Long-Form Storytelling)

สถาปัตยกรรมสำหรับตัวละครที่ไม่ใช่ผู้เล่น (NPC) ในระบบนิเวศเกมหรือสภาพแวดล้อมจำลองหลายตัวละคร ต้องรองรับวงจร Perception-Deliberation-Action Loop:

1. **Perception Filter:** NPC รับข้อมูลเหตุการณ์รอบตัว (Events) และคัดกรองเฉพาะเหตุการณ์ที่มีผลต่อเป้าหมายของตน
2. **Goal Tree Evaluator:** NPC ประเมินเป้าหมายหลัก (Long-term Goals) และเป้าหมายย่อย (Short-term Objectives)
3. **Deliberation Layer:** ใช้ LLM ประมวลผลปฏิกิริยาและบทสนทนาโดยอิงตามบุคลิกและเป้าหมายปัจจุบัน
4. **Action Output:** แสดงผลเป็นทั้งคำพูดและคำสั่งการกระทำในสภาพแวดล้อมจำลอง (เช่น MOVE_TO(Tower), SPOKE("Hello"))

17.1 การสร้างเรื่องราวแบบต่อเนื่อง (Long-Form Story Generation)

การรักษาภาพเส้นเรื่องในระยะยาวใช้วิธีร้อยเรียงลำดับชั้น: Book Arc นำไปสู่ Chapter Outline สลายเป็น Scene Beat และส่งเข้า Active Dialogue Generation โดยมี Story Bible (Lorebook Integration) คอยกำกับกฎเกณฑ์ของเรื่อง

18. ระบบกิ่งก้านการสนทนา การประเมินผล และความปลอดภัย (Branching, Evaluation Frameworks & Security)

โครงสร้างข้อมูลการสนทนาแบบกิ่งก้านจัดเก็บในรูปแบบ Directed Acyclic Graph (DAG) หรือ JSON Tree Structure:

```
{
  "tree_id": "tree_9012",
  "root_node": "node_0001",
  "nodes": {
    "node_0001": {
      "sender": "user",
      "text": "Hello",
      "children": ["node_0002", "node_0003"]
    },
    "node_0002": {
      "sender": "character",
      "text": "Greetings, traveler. (Swipe 1)",
      "parent": "node_0001",
      "children": []
    },
    "node_0003": {
      "sender": "character",
      "text": "Who enters my domain? (Swipe 2)",
      "parent": "node_0001",
      "children": ["node_0004"]
    }
  }
}
```

}
}

18.1 กรอบการประเมินผล AI Roleplay (Evaluation Frameworks)

Framework งานวิจัย	มิติการประเมิน (Evaluation Dimensions)	วิธีการประเมิน (Evaluation Method)	จุดเด่นทางเทคนิค
CharacterBox (NAACL 2025)	Character Fidelity, Storyline Navigation, Multi-agent Interaction	ใช้ CharacterNR (Narrator) และ CharacterRM (Reward Model)	จำลอง Sandbox Multi-agent ลดการพึ่งพา GPT-4 API
Crab Framework (ACL 2025)	Tone Match, Emotional Depth, Scenario Alignment, Role Consistency	ใช้ Reward Model เฉพาะทางชื่อ RoleRM	มีประสิทธิภาพสูงกว่า GPT-4 ในการวัดความแม่นยำของการสวมบทบาท
CoSER Benchmark (ICML 2025)	LifeChoice Accuracy, InCharacter Alignment, GCA Performance	ประเมินการจำลองบทบาทตัวละครตามบริบทจริงในหนังสือ	ชุดประเมินผลครอบคลุมตัวละครกว่า 17,966 ตัว

18.2 สถาปัตยกรรมความปลอดภัย (Security Architecture)

ข้อความและข้อมูลความจำที่ส่งเข้ามาต้องผ่าน Context Injection Classifier เพื่อป้องกันคำสั่งแปลกปลอม จากนั้นส่งเข้า Memory Poisoning Inspector ตรวจสอบการฉีดยาข้อมูลจำเท็จ ก่อนบันทึกเข้าสู่ Isolated User Data Sandbox เพื่อป้องกันการรั่วไหลของข้อมูลข้ามผู้ใช้

19. การวิเคราะห์ต้นทุน ประสิทธิภาพ และสถาปัตยกรรมที่แนะนำ (Cost Optimization & Recommended System Architectures)

การเปรียบเทียบสถาปัตยกรรมระบบ 3 ระดับ สำหรับการสร้างระบบ AI Roleplay ของตนเอง:

องค์ประกอบสถาปัตยกรรม	Level 1: Simple MVP	Level 2: Production System	Level 3: Enterprise Architecture
Target User Base	1 - 100 Users (Hobby/Testing)	100 - 50,000 Users	50,000+ Concurrent Users
Frontend UI	Single Page React / HTML UI	Next.js / Tailwind / PWA	React Native App + Next.js SSR
Backend API Gateway	Python FastAPI / Node.js Express	Node.js NestJS Microservices	Go / Rust High-throughput Gateway
Primary Database	SQLite / Supabase PostgreSQL	PostgreSQL (RDS) + Redis Cache	Distributed CockroachDB + Redis Cluster
Vector DB (RAG)	ChromaDB (Local File)	Qdrant / Pgvector	Milvus / Pinecone

องค์ประกอบสถาปัตยกรรม	Level 1: Simple MVP	Level 2: Production System	Level 3: Enterprise Architecture
			Cluster
LLM Inference Strategy	Direct OpenRouter API	Model Routing (DeepSeek + Sonnet)	Self-Hosted vLLM / TensorRT-LLM
Memory Engine	Context Sliding Window	Hybrid Memory (RAG + Summary)	Multi-tier Memory + Reflection Agents
State Tracking	Basic JSON File	PostgreSQL JSONB State Machine	Event-Driven Kafka State Engine
Caching Layer	Redis Memory Cache	Redis + Provider Prompt Caching	RadixAttention Host KV Cache
Queue / Worker	Simple Asyncio Background Tasks	Celery / RabbitMQ	Apache Kafka Cluster
Monitoring Stack	Console Logging	Grafana + Prometheus + Langfuse	OpenTelemetry + Datadog + Custom Eval
Estimated Cost/Month	\$10 - \$50 USD	\$200 - \$1,500 USD	\$5,000+ USD (Scale Dependent)

20. การนำไปใช้งานอ้างอิงและขั้นตอนระบบ (Reference Implementation & Pseudocode)

ตัวอย่างโครงสร้างข้อมูลและการทำงานของสถาปัตยกรรมระบบในรูปแบบภาษาโปรแกรมจำลอง:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "spec": { "type": "string", "enum": ["chara_card_v2"] },
    "spec_version": { "type": "string", "constant": "2.0" },
    "data": {
      "type": "object",
      "properties": {
        "name": { "type": "string" },
        "description": { "type": "string" },
        "personality": { "type": "string" },
        "scenario": { "type": "string" },
        "first_mes": { "type": "string" },
        "mes_example": { "type": "string" },
        "creator_notes": { "type": "string" },
        "system_prompt": { "type": "string" },
        "post_history_instructions": { "type": "string" },
        "alternate_greetings": {
          "type": "array",
          "items": { "type": "string" }
        },
      },
      "tags": {
        "type": "array",

```

```

        "items": { "type": "string" }
    },
    "creator": {
"type[span_31](start_span)[span_31](end_span)[span_37](start_span)[span_37](end_span)": "string" },
        "character_version": { "type": "string" }
    },
    "required": ["name", "description", "first_mes"]
}
},
"required": ["spec", "spec_version", "data"]
}

```

20.1 ขั้นตอนการทำงานหลักของระบบ (Core System Lifecycle Pseudocode)

```

async def process_roleplay_turn(user_id: str, character_id: str,
user_message_text: str):
    # 1. ดึงข้อมูลตัวละครและสถานะการสนทนาปัจจุบัน
    character = await Database.get_character(character_id)
    conversation = await Database.get_active_conversation(user_id,
character_id)

    # 2. สแกนคีย์เวิร์ด Lorebook ผ่าน Aho-Corasick Engine
    lorebook_entries = LorebookEngine.scan(
        text=user_message_text,
        book=character.lorebook,
        depth=5
    )

    # 3. ดึงความทรงจำระยะยาวจาก Vector DB (Hybrid RAG)
    retrieved_memories = await VectorDB.search(
        query=user_message_text,
        user_id=user_id,
        top_k=3,
        threshold=0.75
    )

    # 4. บีบอัดและบริหารจัดการ Token Budget
    token_budget = CalculateTokenBudget(max_context=16000,
max_output=1000)
    prompt_stack = AssemblePromptStack(
        system_prompt=character.system_prompt,
        persona=character.description,
        lorebook=lorebook_entries,
        memories=retrieved_memories,

```

```

        history=conversation.get_recent_history(limit_tokens=10000),
        user_input=user_message_text,
        post_instructions=character.post_history_instructions,
        budget=token_budget
    )

# 5. ส่งค่าขอประมวลผลไปยัง LLM Inference Engine แบบ Streaming
response_stream = await LLMProvider.generate_stream(
    prompt=prompt_stack,
    temperature=0.9,
    min_p=0.05,
    repetition_penalty=1.1
)

# 6. ประมวลผลหลังการสร้างข้อมูลและส่งผลลัพธ์กลับแบบเรียลไทม์
full_response_text = ""
async for chunk in response_stream:
    sanitized_chunk = PostProcessor.clean_chunk(chunk)
    full_response_text += sanitized_chunk
    await Websocket.send_to_user(sanitized_chunk)

# 7. ทำงานเบื้องหลังแบบ Asynchronous เพื่อบันทึกสถานะและความทรงจำ
TaskQueue.enqueue(AsyncUpdateMemoryAndState(
    user_id=user_id,
    user_msg=user_message_text,
    ai_msg=full_response_text,
    current_state=conversation.world_state
))

return {"status": "success"}

```

21. แผนการพัฒนาที่เหมาะสมที่สุด (Target Build Roadmap & Prioritization)

การจัดลำดับการพัฒนาฟีเจอร์สำหรับระบบเริ่มต้น มีดังนี้:

1. **Must Have (จำเป็นต้องมีในระยะแรก):**
 - Decoupled API Proxy Backend เชื่อมต่อ OpenRouter หรือ DeepSeek
 - Character Card V2 Parser อ่านและฝังไฟล์ PNG
 - Token-Budgeted Sliding Window จัดสรรขนาด Context
 - Post-History Instruction Layer สำหรับสลับคำสั่งควบคุม
 - In-place Message Editing & Regeneration
2. **Should Have (ควรเพิ่มในระยะถัดมา):**
 - Keyword-triggered Lorebook Engine (Aho-Corasick)
 - Conversational Tree Node Database รองรับการ Swipe
 - Basic Hybrid RAG System ผ่าน Vector Database
 - Prompt Caching Integration เพื่อลดต้นทุน

3. Nice to Have (เพิ่มประสิทธิภาพเชิงคุณภาพ):
 - Automated Background Summarization
 - Structured JSON State Extraction
 - Multi-User Persona Management
4. Later (ขยายเมื่อระบบมีสเกลใหญ่):
 - Custom Fine-Tuned Model Engine (LoRA/DPO)
 - RadixAttention Host-Memory KV Caching
 - Autonomous Multi-NPC Agent Sandbox

22. ช่องว่างการวิจัยและตารางแยกแยะหลักฐาน (Research Gaps & Evidence Matrix)

ตารางแยกแยะสถานะความถูกต้องของข้อมูล (Evidence & Research Gap Matrix) ในอุตสาหกรรม AI Roleplay ปัจจุบัน:

หัวข้อวิจัย (Research Domain)	รู้แน่ชัด (FACT)	อนุมานเชิงวิศวกรรมได้ (INFERENCE)	ยังไม่ยืนยัน / สมมติฐาน (SPECULATION)	การทดสอบที่ต้องทำเพิ่ม (Action Required)
Character Card Specs	โครงสร้างไฟล์ PNG V2/V3 tEXt Chunk Encoding Base64 JSON	รูปแบบการแปลง Smart Quotes มีผลต่อ JSON Parser ทั้งหมด	V3 Spec จะกลายเป็นมาตรฐานเดียว โดยยกเลิก V2 ในแพลตฟอร์มหลัก	ทดสอบ Parser กับไฟล์ V3 Native Chunk
Character.AI Stack	ใช้ MixAttention, RadixAttention, Int8 Native QAT, 95% Cache Hit Rate	ใช้อัลกอริทึม Vector Cosine Search ในการทำ User Pinned Memories RAG	สถาปัตยกรรมโมเดลภายในปัจจุบันอิงตาม Llama-3 หรือ Transformer Variant	สังเกตพฤติกรรม Context Degradation ย้อนหลัง
JanitorAI / JanitorLLM	Custom Prompt Limit 500K chars, ปรับปรุง Repetition Loop	JLLM มีพื้นฐานมาจาก Mistral หรือ Llama Fine-tune	JanitorAI จะเปลี่ยนไปใช้ระบบสมัครสมาชิกแบบปิดทั้งหมด	ทดสอบการดึงข้อมูลจาก Proxy API Endpoint
Lorebook Activation	Aho-Corasick Engine, Recursive Scanning Depth, Selective Matching	การสแกนแบบ Recursive Depth เกิน 3 ชั้นส่งผลให้เกิด Latency Spike	โมเดลทุกตัวตอบสนองต่อ Lorebook Priority แบบเดียวกัน	Benchmark วัด Latency ตามความลึก Lorebook
Long-term Memory	Vector Search เพียงอย่างเดียว ขัดแย้งกับความต่อเนื่องทางวรรณกรรม	Hybrid Memory Structure ให้ผลลัพธ์ที่ดีที่สุดในงาน Production	LLM จะสามารถจำความทรงจำได้สมบูรณ์โดยไม่ต้องใช้ฐานข้อมูลภายนอก	เปรียบเทียบ Human Evaluation ระหว่าง RAG vs Pure Context

ผลงานที่อ้างอิง

1. RoleRAG: Enhancing LLM Role-Playing via Graph Guided Retrieval, <https://arxiv.org/html/2505.18541v1> 2. Optimizing AI Inference at Character.AI, <https://blog.character.ai/optimizing-ai-inference-at-character-ai-2/> 3. AI Character Cards (2026): V2 vs V3 Explained, 5 Free Download, <https://honeychat.bot/en/blog/ai-character-cards-guide-v2-v3-2026/> 4. Janitor AI: Architecture,

Features, and How It Works,

<https://www.birchwoodu.org/janitor-ai-architecture-features-and-how-it-works> 5. NovelAI 2026

Review: AI Writing & Anime Image Gen - WeavAI Blog,

<https://weavai.app/blog/en/2026/04/23/novelai-2026-review-ai-writing-anime-image-gen/> 6.

malfoyslastname/character-card-spec-v2 - GitHub,

<https://github.com/malfoyslastname/character-card-spec-v2> 7. Trình Tạo Sách Lore AI (Chub AI)

· Miniapp AI miễn phí - miniapps.ai, <https://miniapps.ai/vi/ai-lore-book-generator-chub-ai> 8. Why

Character.AI Forgets You - persistent memory - KintHAI Blog,

<https://blog.kinhtai.ai/why-character-ai-forgets-you-persistent-memory-architecture/> 9. Import

CCv3 character cards & import and match lorebooks · GitHub,

<https://github.com/m-doughty/LLM-Character> 10. A Novel Configurable Role-Playing LLM with

Assessing Benchmark, <https://aclanthology.org/2025.acl-long.731.pdf> 11. Inference-Friendly

Models With MixAttention - arXiv, <https://arxiv.org/html/2409.15012v1> 12. JanitorAI Review

2026: Features, Pricing & Alternatives - Dupple, <https://dupple.com/reviews/janitorai> 13. World

Info | docs.ST.app - SillyTavern Documentation,

<https://docs.sillytavern.app/usage/core-concepts/worldinfo/> 14. SillyTavern User Guide and

Features | PDF - Scribd,

<https://www.scribd.com/document/874786907/SillyTavern-All-Compressed> 15. The Ultimate

Guide to Janitor AI: Everything You Need to Know (2026),

[https://www.tooljunction.io/blog/the-ultimate-guide-to-janitor-ai-everything-you-need-to-know-20](https://www.tooljunction.io/blog/the-ultimate-guide-to-janitor-ai-everything-you-need-to-know-2026)

26 16. The Absolute Beginner's Guide to Using a Proxy with Janitor Part One,

[https://help.janitorai.com/en/article/the-absolute-beginners-guide-to-using-a-proxy-with-janitor-p](https://help.janitorai.com/en/article/the-absolute-beginners-guide-to-using-a-proxy-with-janitor-part-one-19to7y9/)

art-one-19to7y9/ 17. Changelog - Newsroom - Janitor AI, <https://janitorai.com/news/changelog/>

18. Why Does Character AI Forget Everything? - Kenotic Labs,

<https://kenoticlabs.com/insights/character-ai-memory> 19. Using Chat History to Retrieve More

Relevant Memories for AI Agent, <https://medium.com/gaudiy-ai-lab/b628896107aa> 20.

sillytavern-character-generator/character_card_creator_small.md at,

[https://github.com/cha1latte/sillytavern-character-generator/blob/main/character_card_creator_s](https://github.com/cha1latte/sillytavern-character-generator/blob/main/character_card_creator_small.md)

mall.md 21. Advancing Role-Playing Agents with Role-Aware Reasoning - arXiv,

<https://arxiv.org/html/2506.01748v1> 22. NovelAI vs EPOS-AI: Which AI Writing Tool Fits Your

Novel?, <https://epos-ai.ch/en/blog/novelai-vs-epos-ai.html> 23. Training Customizable

Role-Playing LLMs with Large-Scale ... - arXiv, <https://arxiv.org/html/2501.15427v1> 24. World

Info Guide for Character Creation | PDF | Computing - Scribd,

<https://www.scribd.com/document/948126287/World-Info-Encyclopedia> 25. Coordinating

LLM-Based Persona Simulation of Established Roles, <https://icml.cc/virtual/2025/poster/46115>

26. CharacterBox: Evaluating the Role-Playing Capabilities of LLMs in,

<https://aclanthology.org/2025.naacl-long.323.pdf> 27. jasl/tavern_kit - TavernKit - GitHub,

https://github.com/jasl/tavern_kit